

Islamic University of Technology

Department of Computer Science and Engineering

Design Pattern Assignment

**Using Strategy, Decorator, Factory and Template
Method Patterns Together**

Student Name: Fariya Ahmed

Student ID: 210041103

Course Name: Design Pattern

Course Code: CSE 4851

Date of Submission: 07.06.2026

Contents

1	Scenario Definition	3
1.1	Where Each Pattern Is Used	3
2	Reason for Choosing This Scenario	3
3	Design Overview	4
3.1	Strategy Pattern Design	4
3.2	Decorator Pattern Design	4
3.3	Factory Pattern Design	4
3.4	Template Method Pattern Design	4
4	Project File Structure	4
5	Implementation Explanation	5
6	How the Four Patterns Work Together	6
7	Run Instructions	6
7.1	Linux or Git Bash	6
7.2	Windows PowerShell	6
8	Expected Output	6
9	Repository or Drive Link	7
10	Source Code	7
10.1	Main.java	7
10.2	OrderItem.java	8
10.3	BasicMeal.java	8
10.4	OrderItemDecorator.java	9
10.5	ExtraCheeseDecorator.java	9
10.6	PremiumPackagingDecorator.java	10
10.7	PaymentStrategy.java	10
10.8	BkashPayment.java	10
10.9	CardPayment.java	11
10.10	CashOnDeliveryPayment.java	11
10.11	CustomerOrder.java	12
10.12	PaymentStrategyFactory.java	13
10.13	OrderProcessorFactory.java	14

10.14OrderProcessor.java	14
10.15DeliveryOrderProcessor.java	16
10.16PickupOrderProcessor.java	16
11 Conclusion	16

1. Scenario Definition

The scenario chosen for this assignment is a small **online food order processing system**. In this system, a customer places an order from a restaurant application. The order may be a delivery order or a pickup order. The customer can choose different payment options, such as bKash, card payment, or cash on delivery. The ordered food items can also be customized with extra features, such as extra cheese or premium packaging.

This scenario is suitable for combining the Strategy, Decorator, Factory, and Template Method design patterns because the same order processing flow contains several parts that can vary independently. The payment method may change from one order to another, the food item can be extended with optional additions, the type of processor can be created based on order type, and the complete order processing algorithm follows a fixed sequence of steps.

1.1. Where Each Pattern Is Used

- **Strategy Pattern:** Used to select the payment method at runtime. The system supports bKash, card payment, and cash on delivery without changing the order processing class.
- **Decorator Pattern:** Used to add optional features to food items. For example, a basic burger can be wrapped with an extra cheese decorator and a premium packaging decorator.
- **Factory Pattern:** Used to create objects without exposing the object creation logic to the client code. The project uses factories for payment strategies and order processors.
- **Template Method Pattern:** Used to define the fixed order processing workflow. The main steps are validation, item preparation, service handling, payment collection, and receipt generation. Subclasses only change the service-specific step.

2. Reason for Choosing This Scenario

A food ordering system is easy to understand but still has enough variation to justify the use of multiple design patterns. In a real application, every order follows almost the same business process, but small parts of the process can differ. For example, a delivery order needs rider assignment, while a pickup order only needs to be kept ready at the counter. Similarly, the payment method can vary without affecting the rest of the order processing logic.

The main purpose of the design is to avoid writing one large class with many conditional statements. Instead of putting all cases inside one class, each design pattern separates one kind of responsibility. This makes the code easier to extend, test, and maintain.

3. Design Overview

3.1. Strategy Pattern Design

The `PaymentStrategy` interface defines a common method named `pay()`. Each payment method implements this interface in its own way. The order processor does not need to know the internal details of bKash, card, or cash on delivery. It only calls the common interface method.

3.2. Decorator Pattern Design

The `OrderItem` interface represents a food item. A `BasicMeal` is the simplest implementation of this interface. Decorator classes also implement `OrderItem` and hold a reference to another `OrderItem`. This allows decorators to add new description text and extra cost while keeping the original item unchanged.

3.3. Factory Pattern Design

Two factories are used in this project. `PaymentStrategyFactory` creates a suitable payment strategy based on the selected payment type. `OrderProcessorFactory` creates the correct processor based on the order type. As a result, the `Main` class does not directly depend on all concrete classes.

3.4. Template Method Pattern Design

The abstract class `OrderProcessor` defines the complete order processing algorithm using a final method named `processOrder()`. The steps are fixed, so subclasses cannot accidentally change the order of execution. However, subclasses must implement `handleServiceType()`, because delivery and pickup orders need different handling.

4. Project File Structure

```
1 Design Pattern Assignment/  
2 |-- README.md  
3 |-- report/  
4 | |-- Report.pdf
```

```
5 | -- src/
6 | | -- main/
7 | | -- java/
8 | | -- com/
9 | | -- fariya/
10 | | -- order/
11 | | -- app/
12 | | | -- Main.java
13 | | -- decorator/
14 | | | -- OrderItemDecorator.java
15 | | | -- ExtraCheeseDecorator.java
16 | | | -- PremiumPackagingDecorator.java
17 | | -- factory/
18 | | | -- OrderProcessorFactory.java
19 | | | -- PaymentStrategyFactory.java
20 | | -- model/
21 | | | -- BasicMeal.java
22 | | | -- CustomerOrder.java
23 | | | -- OrderItem.java
24 | | -- strategy/
25 | | | -- PaymentStrategy.java
26 | | | -- BkashPayment.java
27 | | | -- CardPayment.java
28 | | | -- CashOnDeliveryPayment.java
29 | | -- template/
30 | | -- OrderProcessor.java
31 | | -- DeliveryOrderProcessor.java
32 | | -- PickupOrderProcessor.java
```

Listing 1: Project file structure

5. Implementation Explanation

The implementation is written in Java. The Main class creates a customer order, decorates the ordered items, selects a payment strategy, creates the required order processor through a factory, and finally processes the order.

A sample order contains a chicken burger and creamy pasta. The burger is decorated with extra cheese and premium packaging. The pasta is decorated with premium packaging only. The selected payment method is bKash and the selected order type is delivery. These values can be changed easily without modifying the main business workflow.

6. How the Four Patterns Work Together

The four patterns do not work as separate examples in this project. They are connected inside one order processing scenario.

First, the client code uses the **Factory Pattern** to create the payment strategy and the order processor. Then, the ordered food items are modified using the **Decorator Pattern**. After that, the selected **Strategy Pattern** object is attached to the order so that payment can be performed dynamically. Finally, the **Template Method Pattern** runs the complete order processing flow in a fixed order.

This combination makes the design flexible. A new payment method can be added by implementing `PaymentStrategy`. A new food customization can be added by creating a new decorator. A new order type can be added by extending `OrderProcessor`. These changes can be made without rewriting the whole system.

7. Run Instructions

The project can be compiled and run from the root directory using the following commands.

7.1. Linux or Git Bash

```
1 javac -d out $(find src/main/java -name "*.java")
2 java -cp out com.fariya.order.app.Main
```

7.2. Windows PowerShell

```
1 Get-ChildItem -Recurse src/main/java -Filter *.java | ForEach-Object
   { $_.FullName } > sources.txt
2 javac -d out @sources.txt
3 java -cp out com.fariya.order.app.Main
```

8. Expected Output

```
1 Order validated for Fariya Ahmed
2 Preparing ordered items:
3   - Chicken Burger, extra cheese, premium packaging = BDT 320.0
4   - Creamy Pasta, premium packaging = BDT 320.0
5 Assigning a rider for delivery to: IUT Campus, Gazipur
6 Selected payment method: bKash
```

```
7 Paid BDT 640.0 through bKash account 017XXXXXXX
8 Receipt sent. Total bill: BDT 640.0
9 Order completed successfully.
```

Listing 2: Sample output

9. Repository or Drive Link

Code Link: <https://github.com/Fa-riya/Design-Pattern-Assignment>

10. Source Code

The most important source files are included below for reference.

10.1. Main.java

```
1 package com.fariya.order.app;
2
3 import com.fariya.order.decorator.ExtraCheeseDecorator;
4 import com.fariya.order.decorator.PremiumPackagingDecorator;
5 import com.fariya.order.factory.OrderProcessorFactory;
6 import com.fariya.order.factory.PaymentStrategyFactory;
7 import com.fariya.order.model.BasicMeal;
8 import com.fariya.order.model.CustomerOrder;
9 import com.fariya.order.model.OrderItem;
10 import com.fariya.order.strategy.PaymentStrategy;
11 import com.fariya.order.template.OrderProcessor;
12
13 public class Main {
14     public static void main(String[] args) {
15         PaymentStrategy payment = PaymentStrategyFactory.
16             createPaymentStrategy("bkash");
17
18         CustomerOrder order = new CustomerOrder(
19             "Fariya Ahmed",
20             "IUT Campus, Gazipur",
21             payment
22         );
23
24         OrderItem burger = new BasicMeal("Chicken Burger", 220.0);
25         burger = new ExtraCheeseDecorator(burger);
```

```
25     burger = new PremiumPackagingDecorator(burger);
26
27     OrderItem pasta = new BasicMeal("Creamy Pasta", 280.0);
28     pasta = new PremiumPackagingDecorator(pasta);
29
30     order.addItem(burger);
31     order.addItem(pasta);
32
33     OrderProcessor processor = OrderProcessorFactory.
        createOrderProcessor("delivery");
34     processor.processOrder(order);
35 }
36 }
```

10.2. OrderItem.java

```
1 package com.fariya.order.model;
2
3 public interface OrderItem {
4     String getDescription();
5     double getPrice();
6 }
```

10.3. BasicMeal.java

```
1 package com.fariya.order.model;
2
3 public class BasicMeal implements OrderItem {
4     private final String name;
5     private final double price;
6
7     public BasicMeal(String name, double price) {
8         this.name = name;
9         this.price = price;
10    }
11
12    @Override
13    public String getDescription() {
14        return name;
15    }
16 }
```

```
17     @Override
18     public double getPrice() {
19         return price;
20     }
21 }
```

10.4. OrderItemDecorator.java

```
1 package com.fariya.order.decorator;
2
3 import com.fariya.order.model.OrderItem;
4
5 public abstract class OrderItemDecorator implements OrderItem {
6     protected final OrderItem item;
7
8     protected OrderItemDecorator(OrderItem item) {
9         this.item = item;
10    }
11 }
```

10.5. ExtraCheeseDecorator.java

```
1 package com.fariya.order.decorator;
2
3 import com.fariya.order.model.OrderItem;
4
5 public class ExtraCheeseDecorator extends OrderItemDecorator {
6     public ExtraCheeseDecorator(OrderItem item) {
7         super(item);
8     }
9
10    @Override
11    public String getDescription() {
12        return item.getDescription() + ", extra cheese";
13    }
14
15    @Override
16    public double getPrice() {
17        return item.getPrice() + 60.0;
18    }
19 }
```

10.6. PremiumPackagingDecorator.java

```
1 package com.fariya.order.decorator;
2
3 import com.fariya.order.model.OrderItem;
4
5 public class PremiumPackagingDecorator extends OrderItemDecorator {
6     public PremiumPackagingDecorator(OrderItem item) {
7         super(item);
8     }
9
10    @Override
11    public String getDescription() {
12        return item.getDescription() + ", premium packaging";
13    }
14
15    @Override
16    public double getPrice() {
17        return item.getPrice() + 40.0;
18    }
19 }
```

10.7. PaymentStrategy.java

```
1 package com.fariya.order.strategy;
2
3 public interface PaymentStrategy {
4     void pay(double amount);
5     String getPaymentName();
6 }
```

10.8. BkashPayment.java

```
1 package com.fariya.order.strategy;
2
3 public class BkashPayment implements PaymentStrategy {
4     private final String phoneNumber;
5 }
```

```
6    public BkashPayment(String phoneNumber) {
7        this.phoneNumber = phoneNumber;
8    }
9
10   @Override
11   public void pay(double amount) {
12       System.out.println("Paid BDT " + amount + " through bKash
13           account " + phoneNumber);
14   }
15
16   @Override
17   public String getPaymentName() {
18       return "bKash";
19   }
```

10.9. CardPayment.java

```
1 package com.fariya.order.strategy;
2
3 public class CardPayment implements PaymentStrategy {
4     private final String maskedCardNumber;
5
6     public CardPayment(String maskedCardNumber) {
7         this.maskedCardNumber = maskedCardNumber;
8     }
9
10    @Override
11    public void pay(double amount) {
12        System.out.println("Paid BDT " + amount + " using card " +
13            maskedCardNumber);
14    }
15
16    @Override
17    public String getPaymentName() {
18        return "Card";
19    }
```

10.10. CashOnDeliveryPayment.java

```
1 package com.fariya.order.strategy;
2
3 public class CashOnDeliveryPayment implements PaymentStrategy {
4     @Override
5     public void pay(double amount) {
6         System.out.println("Payment of BDT " + amount + " will be
7             collected during delivery.");
8     }
9
10    @Override
11    public String getPaymentName() {
12        return "Cash on Delivery";
13    }
14 }
```

10.11. CustomerOrder.java

```
1 package com.fariya.order.model;
2
3 import com.fariya.order.strategy.PaymentStrategy;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class CustomerOrder {
8     private final String customerName;
9     private final String address;
10    private final List<OrderItem> items;
11    private final PaymentStrategy paymentStrategy;
12
13    public CustomerOrder(String customerName, String address,
14        PaymentStrategy paymentStrategy) {
15        this.customerName = customerName;
16        this.address = address;
17        this.paymentStrategy = paymentStrategy;
18        this.items = new ArrayList<>();
19    }
20
21    public void addItem(OrderItem item) {
22        items.add(item);
23    }
24 }
```

```
23
24 public List<OrderItem> getItems() {
25     return items;
26 }
27
28 public String getCustomerName() {
29     return customerName;
30 }
31
32 public String getAddress() {
33     return address;
34 }
35
36 public PaymentStrategy getPaymentStrategy() {
37     return paymentStrategy;
38 }
39
40 public double getTotalAmount() {
41     return items.stream().mapToDouble(OrderItem::getPrice).sum();
42 }
43 }
```

10.12. PaymentStrategyFactory.java

```
1 package com.fariya.order.factory;
2
3 import com.fariya.order.strategy.BkashPayment;
4 import com.fariya.order.strategy.CardPayment;
5 import com.fariya.order.strategy.CashOnDeliveryPayment;
6 import com.fariya.order.strategy.PaymentStrategy;
7
8 public class PaymentStrategyFactory {
9     public static PaymentStrategy createPaymentStrategy(String
10         paymentType) {
11         if (paymentType == null) {
12             throw new IllegalArgumentException("Payment type cannot be
13                 null.");
14         }
15
16         return switch (paymentType.toLowerCase()) {
17             case "bkash" -> new BkashPayment("017XXXXXXX");
```

```
16         case "card" -> new CardPayment("**** * 4521");
17         case "cod" -> new CashOnDeliveryPayment();
18         default -> throw new IllegalArgumentException("Unsupported
           payment type: " + paymentType);
19     };
20 }
21 }
```

10.13. OrderProcessorFactory.java

```
1 package com.fariya.order.factory;
2
3 import com.fariya.order.template.DeliveryOrderProcessor;
4 import com.fariya.order.template.OrderProcessor;
5 import com.fariya.order.template.PickupOrderProcessor;
6
7 public class OrderProcessorFactory {
8     public static OrderProcessor createOrderProcessor(String
           orderType) {
9         if (orderType == null) {
10             throw new IllegalArgumentException("Order type cannot be
              null.");
11         }
12
13         return switch (orderType.toLowerCase()) {
14             case "delivery" -> new DeliveryOrderProcessor();
15             case "pickup" -> new PickupOrderProcessor();
16             default -> throw new IllegalArgumentException("Unsupported
              order type: " + orderType);
17         };
18     }
19 }
```

10.14. OrderProcessor.java

```
1 package com.fariya.order.template;
2
3 import com.fariya.order.model.CustomerOrder;
4 import com.fariya.order.model.OrderItem;
5
6 public abstract class OrderProcessor {
```

```
7  public final void processOrder(CustomerOrder order) {
8      validateOrder(order);
9      prepareItems(order);
10     handleServiceType(order);
11     collectPayment(order);
12     sendReceipt(order);
13 }
14
15 private void validateOrder(CustomerOrder order) {
16     if (order.getItems().isEmpty()) {
17         throw new IllegalArgumentException("Order must have at
18             least one item.");
19     }
20     System.out.println("Order validated for " + order.
21         getCustomerName());
22 }
23
24 private void prepareItems(CustomerOrder order) {
25     System.out.println("Preparing ordered items:");
26     for (OrderItem item : order.getItems()) {
27         System.out.println(" - " + item.getDescription() + " = BDT
28             " + item.getPrice());
29     }
30 }
31
32 protected abstract void handleServiceType(CustomerOrder order);
33
34 private void collectPayment(CustomerOrder order) {
35     System.out.println("Selected payment method: " + order.
36         getPaymentStrategy().getPaymentName());
37     order.getPaymentStrategy().pay(order.getTotalAmount());
38 }
39
40 private void sendReceipt(CustomerOrder order) {
41     System.out.println("Receipt sent. Total bill: BDT " + order.
42         getTotalAmount());
43     System.out.println("Order completed successfully.\n");
44 }
45 }
```

10.15. DeliveryOrderProcessor.java

```
1 package com.fariya.order.template;
2
3 import com.fariya.order.model.CustomerOrder;
4
5 public class DeliveryOrderProcessor extends OrderProcessor {
6     @Override
7     protected void handleServiceType(CustomerOrder order) {
8         System.out.println("Assigning a rider for delivery to: " +
9             order.getAddress());
10    }
```

10.16. PickupOrderProcessor.java

```
1 package com.fariya.order.template;
2
3 import com.fariya.order.model.CustomerOrder;
4
5 public class PickupOrderProcessor extends OrderProcessor {
6     @Override
7     protected void handleServiceType(CustomerOrder order) {
8         System.out.println("Keeping the order ready at the pickup
9             counter.");
10    }
```

11. Conclusion

This assignment demonstrates how four design patterns can be used together in a single practical scenario. The Strategy Pattern separates payment behavior, the Decorator Pattern adds optional item features, the Factory Pattern centralizes object creation, and the Template Method Pattern controls the order processing algorithm. Together, they produce a cleaner design than a single class filled with many conditional statements.

The final design is open for extension. New payment methods, item add-ons, and order types can be added with small changes, while the core workflow remains stable.